Testing Techniques

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Verification and Validation

- Verification and Validation is the process of investigating that a software system satisfies specifications and standards and it fulfills the required purpose.
- Verification:

  "Are we building the product right".

- The software should conform to its specification.
- Validation:

  "Are we building the right product".

- The software should do what the user really requires.

Available on notes.arpanadhikari7.com.np

Software Testing-Introduction

- Software testing is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect-free in order to produce a quality product.
- Software Testing is a method to check whether the actual software product matches expected requirements and to ensure that software product is Defect free.
- According to ANSI/IEEE 1059 standard – A process of analyzing a software item to detect the differences between existing and required conditions (i.e., defects) and to evaluate the features of the software item.
- The purpose of software testing is to identify errors, gaps or missing requirements in contrast to actual requirements.
- It involves execution of software/system components using manual or automated tools to evaluate one or more properties of interest.

Available on notes.arpanadhikari7.com.np

Error, Bug, Fault and Failure

- Error: It is a human action that produces the incorrect result that produces a fault.
- Bug: The presence of error at the time of execution of the software.
- Fault: State of software caused by an error.
- Failure: Deviation of software from its expected result.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Process Goals

- Validation testing
  - To demonstrate to the developer and the system customer that the software meets its requirements;
  - A successful test shows that the system operates as intended.
- Defect testing
  - To discover faults or defects in the software where its behaviour is incorrect or not in conformance with its specification;
  - A successful test is a test that makes the system perform incorrectly and so exposes a defect in the system.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Need of Software Testing

1. **Cost-Effective:** It is one of the important advantages of software testing. Testing any IT project on time helps you to save your money for the long term. In case if the bugs caught in the earlier stage of software testing, it costs less to fix.
2. **Security:** It is the most vulnerable and sensitive benefit of software testing. People are looking for trusted products. It helps in removing risks and problems earlier.
3. **Product quality:** It is an essential requirement of any software product. Testing ensures a quality product is delivered to customers.
4. **Customer Satisfaction:** The main aim of any product is to give satisfaction to their customers. UI/UX Testing ensures the best user experience.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Artifacts

- Test Artifacts are the deliverables that are given to the stakeholders of a software project.
- Some of the deliverables are provided before the testing phase commences and some are provided during the testing phase and the rest after the testing phase is completed.
- Some of the test deliverables are as follows:
  - Test plan
  - Test case
  - Traceability matrix
  - Test script
  - Test suite
  - Test Fixture and Test harness

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Artifacts-Test plan

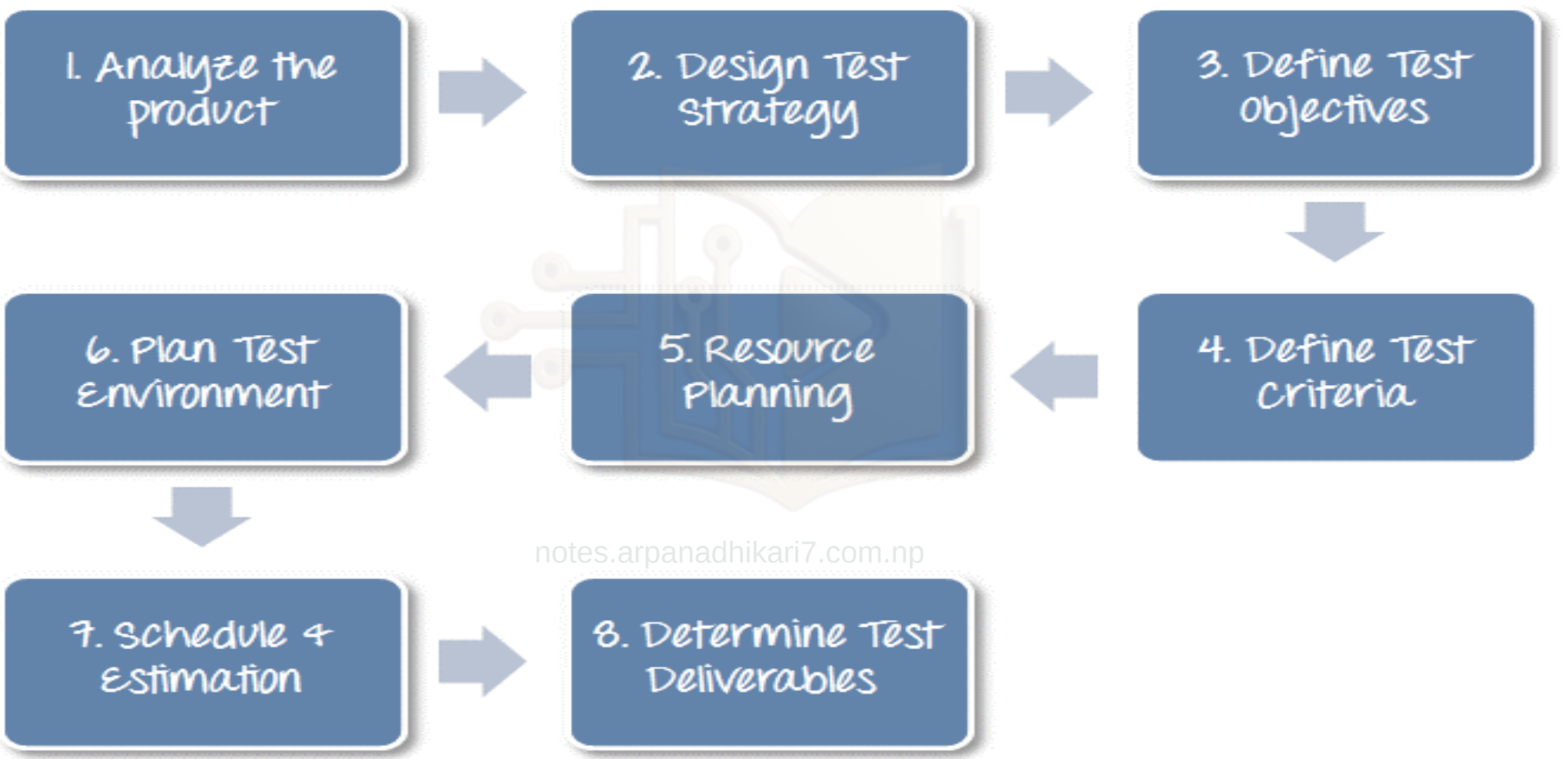
- A Test Plan is a detailed document that describes the
  - Test strategy
  - Objectives
  - Schedule
  - Estimation
  - Deliverables
  - Resources

  required to perform testing for a software product.
- Test Plan helps us determine the effort needed to validate the quality of the application under test.
- The test plan serves as a blueprint to conduct software testing activities as a defined process, which is minutely monitored and controlled by the test manager.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Steps To Create a Test Plan



notes.arpanadhikari7.com.np

Testing Artifacts-Test Case

- A test case is a set of actions executed to verify a particular feature or functionality of your software application.
- A test case is a document, which has a set of test data, preconditions, expected results and post condition, developed for a particular test scenario in order to verify compliance against a specific requirement.
- Test Case acts as the starting point for the test execution, and after applying a set of input values, the application has a definitive outcome and leaves the system at some end point or also known as execution post condition.
- For a Test Scenario: Check Login Functionality there many possible test cases are:
  - Test Case 1: Check results on entering valid User Id & Password
  - Test Case 2: Check results on entering Invalid User ID & Password
  - Test Case 3: Check response when a User ID is Empty & Login Button is pressed, and many more

Available on notes.arpanadhikari7.com.np

The Format of Standard Test Cases

Test Case ID	Test Scenario	Test Steps	Test Data	Observed Results	Actual Results	Pass/Fail
TU01	Check Customer Login with valid Data	1.Go to site 2.Enter UserId 3.Enter Password 4.Click Submit	Userid = abcd Password = pass123	User should Login into an application	As Expected	Pass

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Artifacts-Traceability Matrix

- Traceability matrix is a document that traces and maps the relationship between two baseline documents.
- **Requirement Traceability Matrix(RTM)** traces and maps the requirement and the test cases.
- The main purpose of Requirement Traceability Matrix is to validate that all requirements are checked via test cases such that no functionality is unchecked during Software testing.
- RTM basically contains:
  1. Requirement ID
  2. Requirement Type and Description
  3. Test Cases with Status

Available on notes.arpanadhikari7.com.np

Requirement Traceability Matrix(RTM) Sample

Req No	Req Desc	Testcase ID	Status
123	Login to the application	TC01,TC02,TC03	TC01-Pass TC02-Pass
345	Ticket Creation	TC04,TC05,TC06, TC07,TC08,TC09 TC010	TC04-Pass TC05-Pass TC06-Pass TC06-Fail TC07-No Run
456	Search Ticket	TC011,TC012, TC013,TC014	TC011-Pass TC012-Fail TC013-Pass TC014-No Run

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Artifacts-Test script

- A Test Script is a set of instructions that is performed on a system under test to verify that the system performs as expected.
- They can be written in either a human language (used for manual testing) or a scripting / programming language (used for automated testing).
- A test script is a part of a test case and a test case can have either a single test script or multiple test scripts.
- Some scripting languages used in automated testing are:
  - JavaScript
  - Perl
  - Python
  - Ruby
  - Unix Shell Script
  - VBScript

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Artifacts-Test suite

- A Test Suite is a collection of test cases.
- In automated testing, it can mean a collection of test scripts.
- In a test suite, the test cases / scripts are organized in a logical order.
- For example, the test case for registration will precede the test case for login.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Artifacts-Test Fixture and Test Harness

- A test fixture is an environment used to consistently test some item, device, or piece of software.
- Test Harness in Software Testing is a **collection of stubs, drivers and other supporting tools required to automate test execution.**
- Test harness executes tests by using a test library and generates test reports.
- Test harness contains all the information needed to compile and run a test like test cases, target deployment port(TDP), source file under test, stubs, etc.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Entry and Exit Criteria

➢Entry Criteria

- It can be defined as the fulfilment of minimum set of conditions to begin with the process of testing work.
- The various conditions may be installation and configuration of hardware and software applications as well.

➢Exit Criteria

- To close the project, all necessary tasks should have met by now.
- All test plans have been executed, requirement coverage has been done, and all the risk affected areas have been covered.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types of Software Testing

1. Manual Testing
2. Automation Testing

notes.arpanadhikari7.com.np

Types of Software Testing-Manual Testing

- Manual Testing is a type of software testing in which test cases are executed manually by a tester without using any automated tools.
- Manual Testing is performed by a human sitting in front of a computer carefully executing the test steps.
- The purpose of Manual Testing is to identify the bugs, issues, and defects in the software application.
- Manual software testing is the most primitive technique of all testing types and it helps to find critical bugs in the software application.
- Any new application must be manually tested before its testing can be automated.
- Manual Testing concepts does not require knowledge of any testing tool.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Manual Testing-Need of Manual Testing

1. **Human Perspective:** The basic usability and look & feel of the application can only be gazed and evaluated by Humans. As the software is developed for humans only, so they only can do better justice of validation from a user experience perspective.
2. **A broader perspective and variation of the System workflows:** Manual verification always gives a broader perspective of the overall application. It will provide more expansive coverage for the system validation.
3. **Un-automatable scenarios:** There can be multiple scenarios that are either not worth automating and doesn’t give clear confidence of the user behavior when just testing using automation.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

How To Perform Manual Testing

1. Read and understand the software project documentation/guides. Also, study the Application Under Test (AUT) if available.
2. Draft Test cases that cover all the requirements mentioned in the documentation.
3. Review and baseline the test cases with Team Lead, Client (as applicable)
4. Execute the test cases on the AUT
5. Report bugs
6. Once bugs are fixed, again execute the failing test cases to verify they pass

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Types of Software Testing-Automation Testing

- Automation Testing or Test Automation is a software testing technique that performs test using special automated testing software tools to execute a test case suite.
- In this process, testers execute the test scripts and generate the test results automatically by using automation tools.
- The automation testing software can also enter test data into the System Under Test, compare expected and actual results and generate detailed test reports.
- Test Automation demands considerable investments of money and resources.
- Using a test automation tool, it's possible to record the test suite and re-play it as required.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Why Test Automation?

- Manual Testing of all workflows, all fields, all negative scenarios is time and money consuming
- It is difficult to test for multilingual sites manually
- Test Automation does not require Human intervention. You can run automated test unattended (overnight)
- Test Automation increases the speed of test execution
- Automation helps increase Test Coverage
- Manual Testing can become boring and hence error-prone.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Automated Testing Process

Step 1) Test Tool Selection

Step 2) Define scope of Automation: The scope of automation is the area of our Application Under Test which will be automated

Step 3) Planning, Design and Development: During this phase, we create an Automation strategy and plan, which contains the following details

Step 4) Test Execution: Automation Scripts are executed during this phase. The scripts need input test data before there are set to run. Once executed they provide detailed test reports.

Step 5) Maintenance

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Automation Testing Tools

1. Ranorex Studio: for automating functional UI tests, **regression tests (test to ensure change introduces no new defects)**, data-driven tests and much more.
2. Selenium: software testing tool used for Regression Testing
3. QTP: QTP is widely used for functional and regression testing, it addresses every major software application and environment.
4. jMeter: It is designed to cover various categories of tests such as load testing, functional testing, performance testing, regression testing, etc
5. SilkTest
6. TestComplete
7. WinRunner
8. WATIR

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Testing: Testing Methods

1. Static Testing
2. Dynamic Testing

notes.arpanadhikari7.com.np

Testing Methods-Static Testing

- Static Testing, a software testing technique in which the software is tested without executing the code.
- Static testing is done to avoid errors at an early stage of development as it is easier to identify the errors and solve the errors
- It is also known as Verification in Software Testing.
- Verification is a static method of checking documents and files.
- Verification is the process, to ensure that whether we are building the product right i.e., to verify the requirements which we have and to verify whether we are developing the product accordingly or not.

Available on notes.arpanadhikari7.com.np

Testing Methods-Dynamic Testing

- Dynamic Testing is a software testing method used to test the dynamic behavior of software code.
- The main purpose of dynamic testing is to test software behavior with dynamic variables or variables which are not constant and finding weak areas in software runtime environment.
- The code must be executed in order to test the dynamic behavior.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

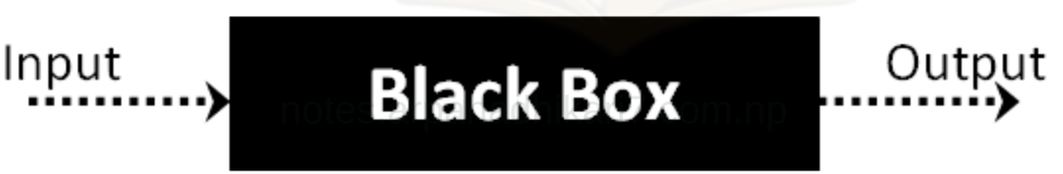
Testing Approaches

1. Black Box Testing
2. White Box Testing
3. Grey Box Testing

notes.arpanadhikari7.com.np

Black Box Testing

- Black Box Testing is a software testing method in which testers evaluate the functionality of the software under test without looking at the internal code structure.
- Black Box Testing mainly focuses on input and output of software applications and it is entirely based on software requirements and specifications.
- It is also known as Behavioral testing.



Available on notes.arpanadhikari7.com.np

Steps Involved In Performing Black Box Testing

1. Initially, the requirements and specifications of the system are examined.
2. Tester chooses valid inputs (positive test scenario) to check whether SUT processes them correctly. Also, some invalid inputs (negative test scenario) are chosen to verify that the SUT is able to detect them.
3. Tester determines expected outputs for all those inputs.
4. Software tester constructs test cases with the selected inputs.
5. The test cases are executed.
6. Software tester compares the actual outputs with the expected outputs.
7. Defects if any are fixed and re-tested

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Prominent Types of Black Box Testing

1. Functional Testing
2. Non-Functional Testing
3. Regression Testing

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Prominent Types of Black Box Testing-Functional Testing

- Functional testing type is related to the functional requirements of a system; it is done by software testers.
- In simple words, what the system actually does is functional testing.
- Functional Testing is used to verify that each function of the software application behaves as specified in the requirement document.
- Testing all the functionalities by providing appropriate input to verify whether the actual output is matching the expected output or not.
- It falls within the scope of black-box testing and the testers need not concern about the source code of the application.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Prominent Types of Black Box Testing-Non-Functional Testing

- This type of black box testing is not related to testing of specific functionality, but non-functional requirements such as performance, scalability, usability.
- In simple words, how well the system performs is non-functionality testing.
- The main focus is to improve the user experience on how fast the system responds to a request.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Prominent Types of Black Box Testing-Regression Testing

- Regression Testing is defined as a type of software testing to confirm that a recent program or code change has not adversely affected existing features.
- This testing is done to make sure that new code changes should not have side effects on the existing functionalities.
- The need of Regression Testing mainly arises whenever there is requirement to change the code and we need to test whether the modified code affects the other part of software application or not.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

White Box Testing

- White Box Testing is software testing technique in which internal structure, design and coding of software are tested to verify flow of input-output and to improve design, usability and security.
- In white box testing, **code is visible to testers** so it is also called Clear box testing, Open box testing, Transparent box testing, Code-based testing and Glass box testing.
- The clear box or White Box name symbolizes the ability to see through the software's outer shell (or "box") into its inner workings.
- In White Box Testing we verify:
  - Internal security holes
  - Broken or poorly structured paths in the coding processes
  - The flow of specific inputs through the code
  - Expected output
  - The functionality of conditional loops
  - Testing of each statement, object, and function on an individual basis

Available on notes.arpanadhikari7.com.np

Steps Involved In Performing White Box Testing

Step 1) Understand The Source Code

- Since white box testing involves the testing of the inner workings of an application, the tester must be very knowledgeable in the programming languages used in the applications they are testing.
- Also, the testing person must be highly aware of secure coding practices. Security is often one of the primary objectives of testing software.

Step 2) Create Test Cases And Execute

- The second basic step to white box testing involves testing the application's source code for proper flow and structure.
- The tester will develop little tests for each process or series of processes in the application.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Grey Box Testing

- Grey Box Testing is a software testing technique to test a software product or application with partial knowledge of internal structure of the application.
- Grey box is the combination of both White Box and Black Box Testing.
- The purpose of grey box testing is to **search and identify the defects due to improper code structure or improper use of applications.**
- It combines the input of developers as well as testers and improves overall product quality
- It reduces the overhead of long process of testing functional and non-functional types

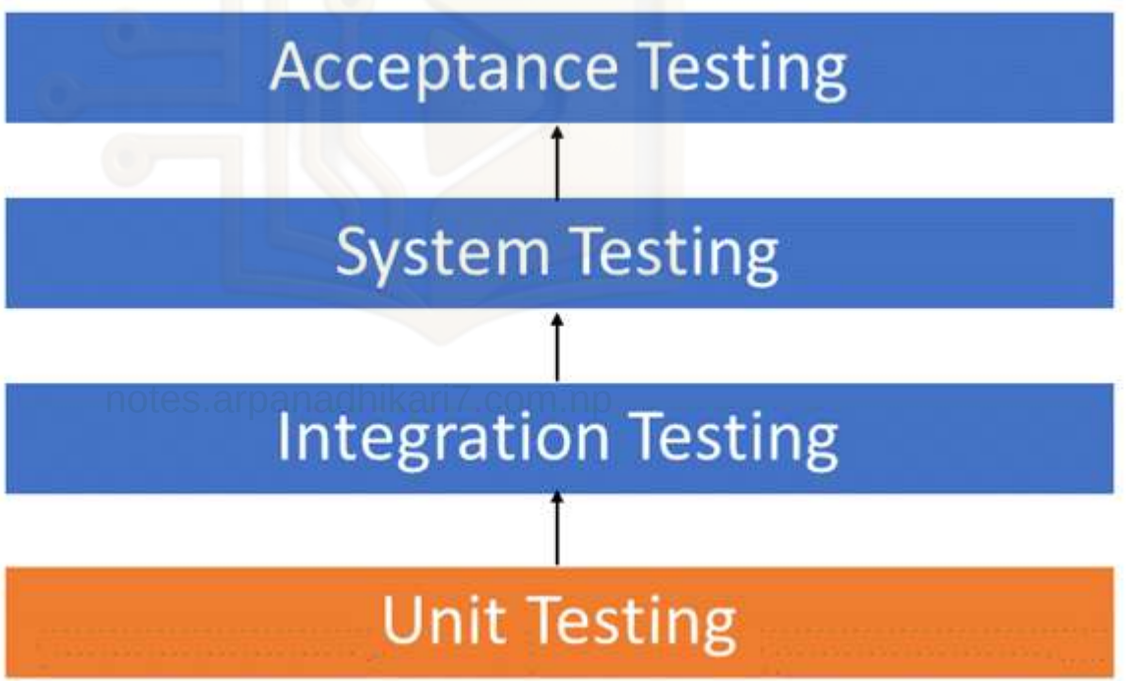


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Levels/Strategies

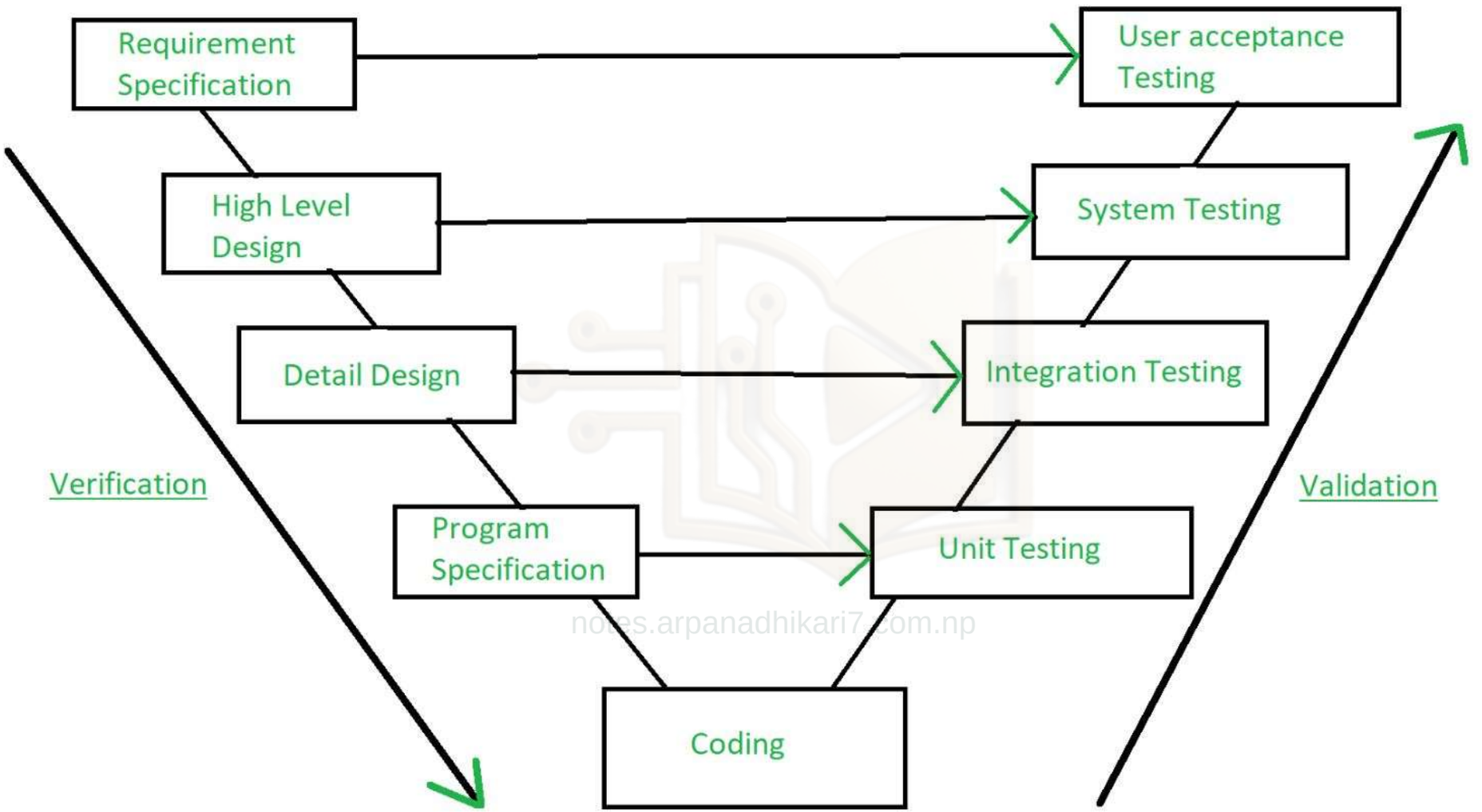
1. Unit Testing
2. Integration Testing
3. System Testing
4. Acceptance Testing



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Testing Levels



Note: Verification is followed by Validation

Available on notes.arpanadhikari7.com.np

Unit Testing

- Unit Testing is a type of software testing where individual units or components of a software are tested.
- A unit may be an individual function, method, procedure, module, or object.
- The purpose is to validate that each unit of the software code performs as expected.
- Unit Testing is done during the development (coding phase) of an application by the developers.
- Unit testing is a White Box testing technique that is usually performed by the developer.
- Unit testing is first level of testing done before integration testing

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

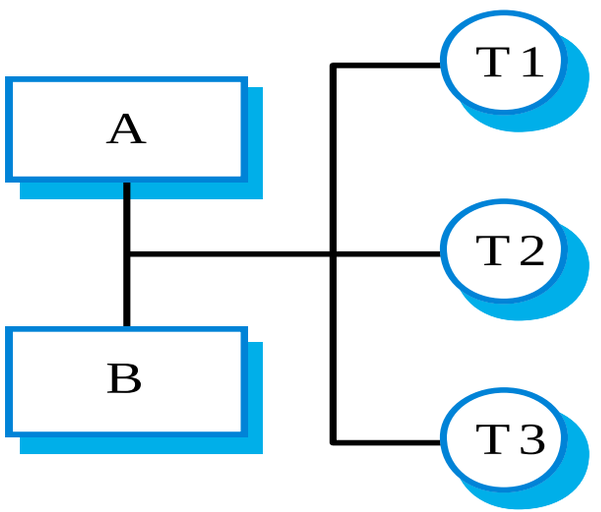
Integration/Component Testing

- Integration Testing is defined as a type of testing where software modules are integrated logically and tested as a group.
- A typical software project consists of multiple software modules, coded by different programmers.
- The purpose of this level of testing is to expose defects in the interaction between these software modules when they are integrated.
- Integration Testing focuses on checking data communication amongst these modules.
- It is subdivided into the Top-Down Approach, Bottom-Up Approach, and Sandwich Approach (Combination of Top-Down and Bottom-Up).
- To simplify error localisation, systems should be incrementally integrated.

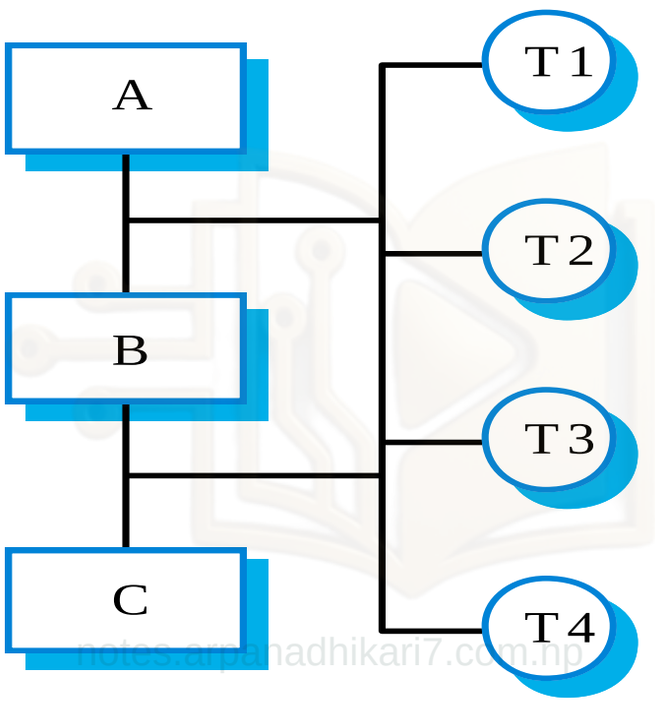
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

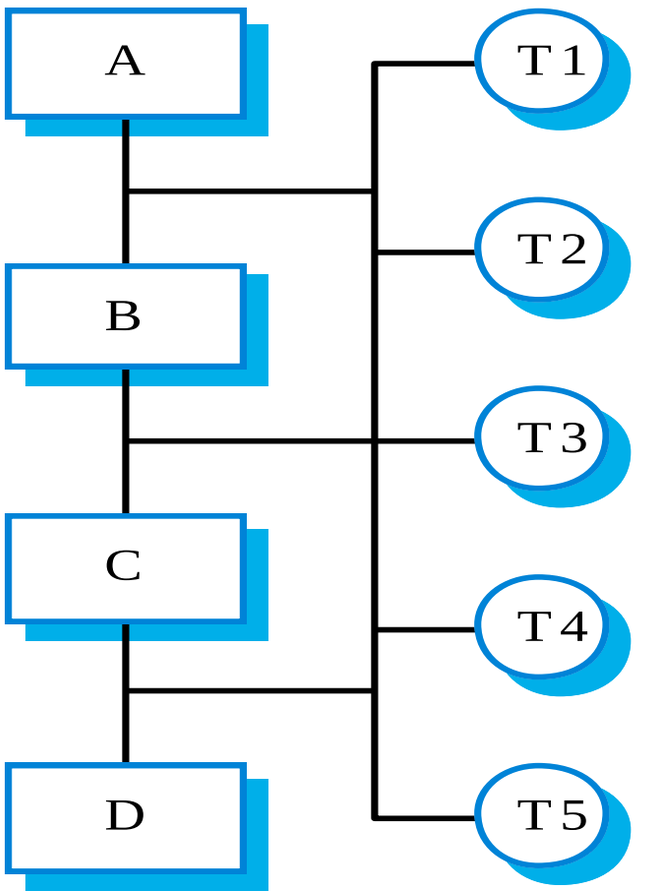
Incremental Integration Testing



Test sequence 1



Test sequence 2



Test sequence 3

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

System Testing

- It’s a black box testing.
- System Testing is a level of testing that validates the complete and fully integrated software product.
- The purpose of a system test is to evaluate the end-to-end system specifications.
- Testing the fully integrated applications including external peripherals in order to check how components interact with one another and with the system as a whole. This is also called End to End testing scenario.
- It also involves testing of the user’s experiences with the application.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Acceptance Testing/User Acceptance Testing (UAT)

- **User Acceptance Testing (UAT)** is a type of testing performed by the end user or the client to verify/accept the software system before moving the software application to the production environment.
- UAT is done in the final phase of testing after unit, integration and system testing is done.
- The main purpose of UAT is to validate end to end business flow. It does not focus on cosmetic errors, spelling mistakes or system testing. User Acceptance Testing is carried out in a separate testing environment with production-like data setup.
- It is kind of black box testing where two or more end-users will be involved.
- Types of Acceptance testing:

1. Alpha Testing
2. Beta Testing
3. Gamma Testing

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Alpha Testing

- Alpha Testing is a type of acceptance testing; performed to identify all possible issues and bugs before releasing the final product to the end users
- Alpha testing is carried out by the testers who are internal employees of the organization.
- The main goal is to identify the tasks that a typical user might perform and test them.

notes.arpanadhikari7.com.np

Beta Testing

- Beta Testing is performed by "real users" of the software application in "real environment" and it can be considered as a form of external User Acceptance Testing.
- It is the final test before shipping a product to the customers.
- Direct feedback from customers is a major advantage of Beta Testing.
- This testing helps to test products in customer's environment.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Alpha Testing vs Beta Testing-Key differences

- Alpha Testing is performed by the Testers within the organization whereas Beta Testing is performed by the end users.
- Alpha Testing is performed at Developer's site whereas Beta Testing is performed at Client's location.
- Alpha Testing involves both White box and Black box testing whereas Beta Testing mainly involves Black box testing.
- Alpha Testing requires testing environment while Beta Testing doesn't require testing environment.
- Alpha Testing requires long execution cycle whereas Beta Testing requires only few weeks of execution.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Gamma Testing

- This mainly focuses on Security checks and Performance checks of the product along with its usability.
- This is the final phase of testing and is performed when the product is ready for release with specific requirements.
- Product in this phase is generally called as Release Candidate or Gamma or Delta, and this is the activity that is done just before the Production launch.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Static And Dynamic Verification

- Software inspections. Concerned with analysis of the static system representation to discover problems (static verification)
  - May be supplement by tool-based document and code analysis
- Software testing. Concerned with exercising and observing product behaviour (dynamic verification)
  - The system is executed with test data and its operational behaviour is observed

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

The V & V process

- It is a whole life-cycle process i.e. V & V must be applied at each stage in the software process.
- Has two principal objectives
  - The discovery of defects in a system;
  - The assessment of whether or not the system is useful and useable in an operational situation.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Verification Process

- Verification is the process of checking that a software achieves its goal without any bugs.
- It is the process to ensure whether the product that is developed is right or not.
- It verifies whether the developed product fulfills the requirements that we have.
- Verification is Static Testing.
- Activities involved in verification:
    1. Inspections
    2. Reviews
    3. Walkthroughs
    4. Desk-checking

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

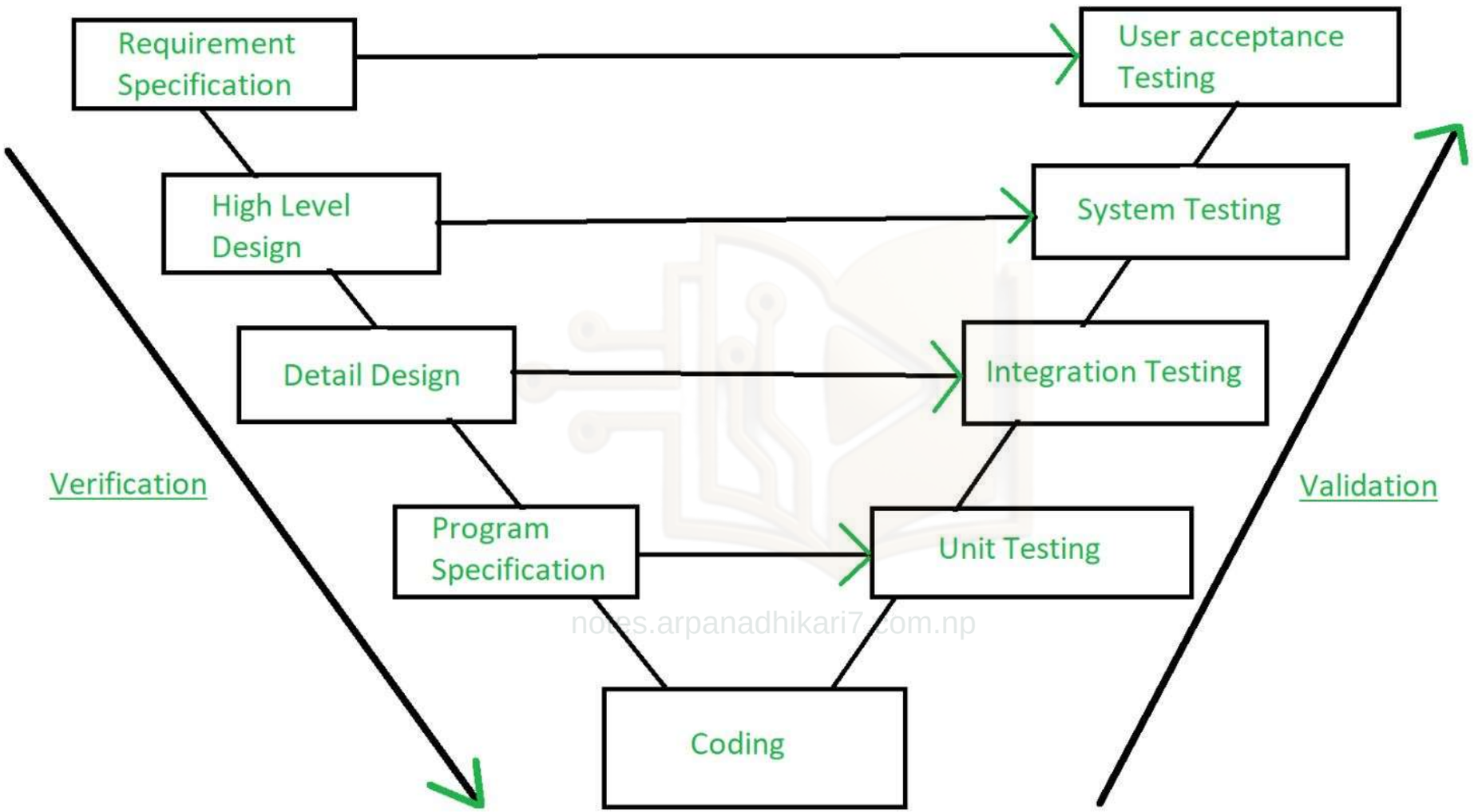
Validation Process

- Validation is the process of checking whether the software product is up to the mark or in other words product has high level requirements.
- It is the process of checking the validation of product i.e. it checks what we are developing is the right product.
- It is validation of actual and expected product.
- Validation is the **Dynamic Testing**
- Activities involved in validation:
    1. Black box testing
    2. White box testing
    3. Unit testing
    4. Integration testing

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

V&V Testing



Note: Verification is followed by Validation

Available on notes.arpanadhikari7.com.np

Verification vs Validation

Verification	Validation
Answers for “Are we building the product right?”	Answers for “Are we building the right product?”
Verification is a static practice of verifying documents, design, code and program.	Validation is a dynamic mechanism of validating and testing the actual product.
It involves evaluating the product in the development phase.	It involves evaluating products at the closing of the development phase.
It does not involve executing the code	It always involves executing the code
Verification uses methods like reviews, walkthroughs, inspections, and desk-checking etc.	It uses methods like Black Box Testing, White Box Testing, Unit testing, Integration testing

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Verification vs Validation

Verification	Validation
It finds bugs early in the development cycle.	It can find bugs that the verification process can not catch.
QA team does verification and make sure that the software is as per the requirement in the SRS document.	With the involvement of testing team validation is executed on software code.
It comes before validation	It comes after verification

notes.arpanadhikari7.com.np

V & V planning

- Careful planning is required to get the most out of testing and inspection processes.
- Planning should start early in the development process.
- The plan should identify the balance between static verification and testing.
- Test planning is about defining standards for the testing process rather than describing product tests.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Steps In Validation Planning

Step 1: Create the Validation Plan: First step is to create a validation plan usually consists of a description of the system, environment specifications, assumptions, limitations, testing criteria, acceptance criteria, identification of the validation team including the responsibilities of each individual, required procedures, and required documentation.

Step 2: Define System Requirements: This steps deals with defining the system requirements that include both functional and non-functional requirements.

Step 3: Create the Validation Protocol & Test Specifications: The testing phase begins with the development of a test plan (VP-Validation Protocol) and test cases (Test Specifications). The test plan describes the objectives, scope, approach, risks, resources, and schedule of the software test. The test cases identify inputs, actions, or events and expected responses to determine if a feature of an application is performing as necessary.

Available on notes.arpanadhikari7.com.np

Steps In Validation Planning

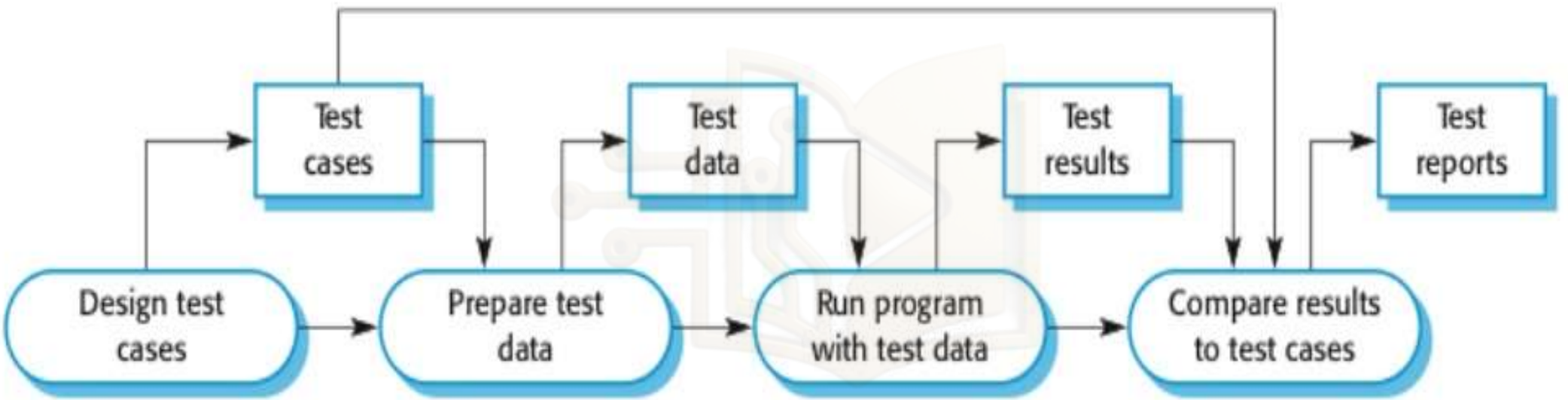
Step 4: Testing: The actual testing is then ready to be initiated. Tests are executed based on the test plan and test cases. Any errors, defects, deviations, and failures are identified and recorded, and are dispositioned in a final report.

Step 5: Develop/Revise Procedures and Final Report: Once testing is completed, procedures for system use and administration must be developed/revised. Then, prior to the system being released for use, a final validation report is produced, reviewed, and approved. The Final Report or Validation Report (VR) typically serves as a validation wrap up.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

The Software Testing Process



notes.arpanadhikari7.com.np

Phases in Software Testing Life Cycle

1. Test Strategy and Test Plan
2. Test Design
3. Test Execution
4. Test Closure

notes.arpanadhikari7.com.np

Phases in Software Testing Life Cycle: Test Strategy and Test Plan

- Every project needs a Test Strategy and a Test Plan.
- Without a clear test strategy and a detailed test plan, even Agile projects will find it difficult to be productive.
- A senior Quality Assurance Manager will be involved at this stage to determine the cost estimates for the project.
- Deliverables of this stage are Test Plan or Strategy Document, Effort Estimation Document.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Phases in Software Testing Life Cycle: Test Design

- Now that we have a strategy and a plan, the next step is to dive into creating a test suite.
- A test suite is a collection of test cases that are necessary to validate the system being built, against its original requirements.
- Also the test data is prepared in this phase.
- At the end of this phase, test environment ready with test data set up

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Phases in Software Testing Life Cycle: Test Execution

- We can execute tests in many different ways-as single, waterfall SIT (System Integration Test) and UAT (User Acceptance Test) phases; as part of Agile sprints; supplemented with exploratory tests; or with test-driven development.
- This involves a thorough testing of the Software, yes, but also a recording of the test results at every point of the execution process.
- Here we record the defects, where they arose, test results at every point of the execution step.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Phases in Software Testing Life Cycle: Test Closure

- Now we need to green-light our product for release.
- We need to consider the exit criteria for signaling completion of the test cycle and readiness for a release.
- The components of exit criteria are:
  - 100% requirements coverage: all business and technical requirements have to be covered by testing.
  - Minimum % pass rate: targeting 90% of all test cases to be passed is best practice.
  - All critical defects to be fixed: self-explanatory. They are critical for a reason.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Development Testing (DevTest)

- Development testing (DevTest) is an approach in software development that aims to bring the development and testing phases closer together.
- Development testing includes all the testing activities that are carried out by the team developing the system.
- In traditional software development, development and testing are two separate functions.
- The challenge with this approach is that it introduces delay between code being written and that same code being tested.
- In DevTest, these phases are more tightly integrated so that code that is being written and checked in is automatically tested.
- In this way, problems can be more quickly discovered and addressed.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Development Testing (DevTest)

➢Development testing (DevTest) involves :

1. Unit Testing: Individual units are tested
2. Integration Testing: Here several units are integrated to create components and these components are tested.
3. System Testing: system as a whole is tested.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

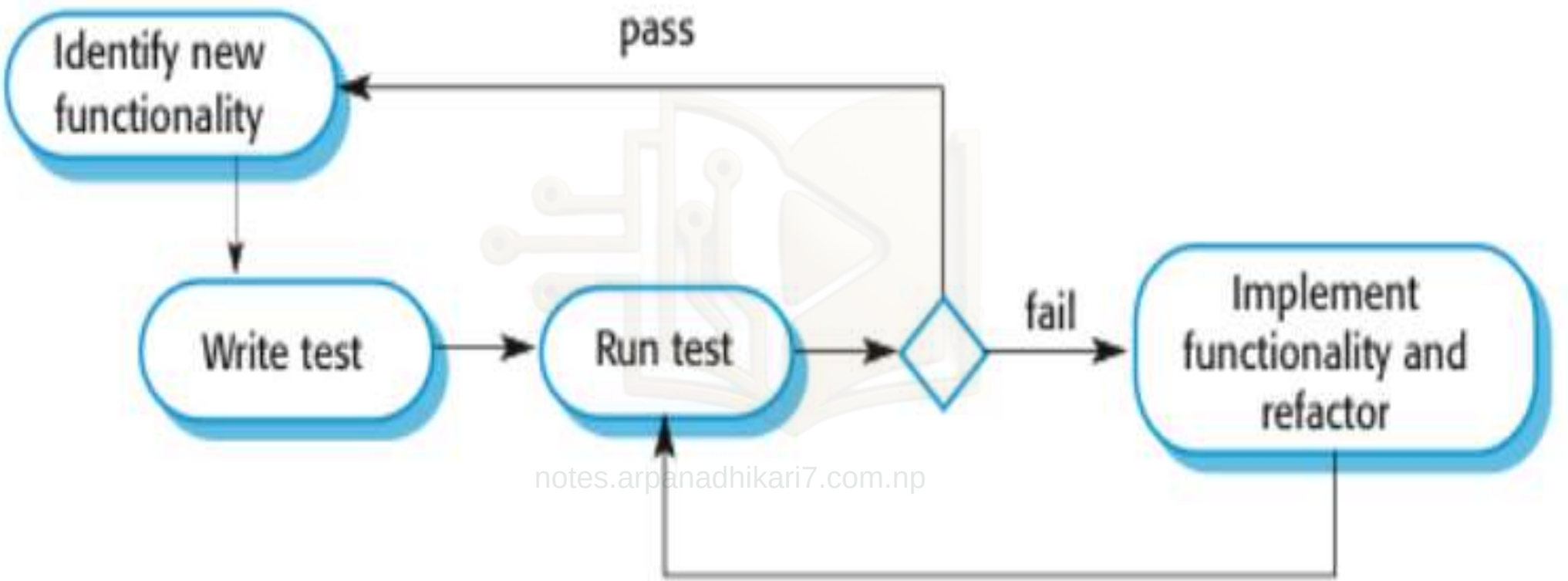
Test-Driven Development(TDD)

- Test Driven Development (TDD) is software development approach in which test cases are developed to specify and validate what the code will do.
- The simple concept of TDD is to write and correct the failed tests before writing new code (before development).
- This means:
  1. test cases for each functionality are created and tested first
  2. if the test fails then the new code is written in order to pass the test and making code simple and bug-free.
- Test-Driven Development starts with designing and developing tests for every small functionality of an application.
- Test Driven Development (TDD) avoids duplication of code.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Test-Driven Development-Flowchart



notes.arpanadhikari7.com.np

Activate Windows

Acceptance TDD And Developer TDD

1. **Acceptance TDD (ATDD):** With ATDD we write a single acceptance test. This test fulfills the requirement of the specification or satisfies the behavior of the system. After that we write just enough production/functionality code to fulfill that acceptance test. Acceptance test focuses on the overall behavior of the system. ATDD also was known as **Behavioral Driven Development (BDD).**
2. **Developer TDD:** With Developer TDD you write single developer test i.e. unit test and then just enough production code to fulfill that test. The unit test focuses on every small functionality of the system. **Developer TDD is simply called as TDD**

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Benefits of TDD

1. Early bug notification
2. Much less debug time
3. Code proven to meet requirements
4. Tests become Safety Net
5. Near zero defects
6. Shorter development cycles

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Release Testing

- Release testing is the process of testing a particular release of a system that is intended for use outside of the development team.
- The primary goal of the release testing process is to convince the customer of the system that it is good enough for use.
- Release testing, therefore, has to show that the system delivers its specified functionality, performance and dependability, and that it does not fail during normal use.
- Release testing is usually a black-box testing process where tests are only derived from the system specification.
- Release testing is a form of system testing.

Available on notes.arpanadhikari7.com.np

Release Testing And System Testing

- Important differences:
  - A separate team that has not been involved in the system development, should be responsible for release testing.
  - System testing by the development team should focus on discovering bugs in the system (defect testing). The objective of release testing is to check that the system meets its requirements and is good enough for external use (validation testing).

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

User Testing

- User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.
- User testing is essential, even when comprehensive system and release testing have been carried out.
- The reason for this is that influences from the user's working environment have a major effect on the reliability, performance, usability and robustness of a system.
- In most software development methodology, the user/customer is part of the development team and is responsible for making decisions on the acceptability of the system.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Performance Testing

- **Performance Testing** is a type of software testing that ensures software applications to perform properly under their expected workload.
- It is a testing technique carried out to determine system performance in terms of sensitivity, reactivity and stability under a particular workload.
- **Performance Testing Attributes:**
  - **Speed:** It determines whether the software product responds rapidly.
  - **Scalability:** It determines amount of load the software product can handle at a time.
  - **Stability:** It determines whether the software product is stable in case of varying workloads.
  - **Reliability:** It determines whether the software product is secure or not.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Performance Testing-Types

1. Load testing: It checks the product's ability to perform under anticipated user loads. The objective is to identify performance congestion before the software product is launched in market.
2. Stress testing: It involves testing a product under extreme workloads to see whether it handles high traffic or not. The objective is to identify the breaking point of a software product.
3. Endurance testing: It is performed to ensure the software can handle the expected load over a long period of time.
4. Spike testing: It tests the product's reaction to sudden large spikes in the load generated by users.
5. Volume testing: In volume testing large number of data is saved in a database and the overall software system's behavior is observed. The objective is to check product's performance under varying database volumes
6. Scalability testing: In scalability testing, software application's effectiveness is determined in scaling up to support an increase in user load. It helps in planning capacity addition to your software system

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Usability Testing

- Usability Testing also known as User Experience(UX) Testing, is a testing method for measuring how easy and user-friendly a software application is.
- Usability testing mainly focuses on user's ease of using application, flexibility of application to handle controls and ability of application to meet its objectives.
- This testing is recommended during the initial design phase of SDLC, which gives more visibility on the expectations of the users.
- There are two methods available to do usability testing -

1. Laboratory Usability Testing: This testing is **conducted in a separate lab room** in presence of the observers. The testers are assigned tasks to execute. The role of the observer is to monitor the behavior of the testers and report the outcome of testing.
2. Remote Usability Testing: Under this testing observers and testers are remotely located. Testers access the System Under Test, remotely and perform assigned tasks. Tester's voice , screen activity , testers facial expressions are recorded by an automated software. Observers analyze this data and report findings of the test.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Test Case Development Strategies

- Test case development strategies are structured approaches used to design effective test cases that ensure the quality and correctness of a software application. Some of the strategies are:
- Boundary Value Analysis
- Equivalence partitioning
- Basis Path Testing
- Control Structure Testing

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Boundary Value Analysis

- **Boundary Value Analysis (BVA)** is a software testing technique used to identify errors at the boundaries of input domains rather than within the range of input values.
- This method is based on the premise that errors are more likely to occur at the edges of input ranges.
- BVA is widely used to design test cases, especially for systems with numerical or ordered input values.
- It is Black box testing.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Boundary Value Analysis

- A boundary value for a valid partition is a valid boundary value.
- A boundary value for an invalid partition is an invalid boundary value.
- For each variable we check-
  - Minimum value.
  - Just above the minimum.
  - Nominal Value.
  - Just below Max value.
  - Max value

notes.arpanadhikari7.com.np

Boundary Value Analysis

Example: Consider a system that accepts ages from 18 to 56.

Boundary Value Analysis(Age accepts 18 to 56)		
Invalid (min-1)	Valid (min, min + 1, nominal, max – 1, max)	Invalid (max + 1)
17	18, 19, 37, 55, 56	57

- **Valid Test cases:** Valid test cases for the above can be any value entered greater than 17 and less than 57.

- Enter the value- 18.
- Enter the value- 19.
- Enter the value- 37.
- Enter the value- 55.
- Enter the value- 56.

notes.arpanadhikari7.com.np

Boundary Value Analysis

- **Invalid Testcases:** When any value less than 18 and greater than 56 is entered.

  - Enter the value- 17.

  - Enter the value- 57.

- **Single Fault Assumption:** When more than one variable for the same application is checked then one can use a single fault assumption. Holding all but one variable to the extreme value and allowing the remaining variable to take the extreme value.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Equivalence Partitioning

- **Equivalence Partitioning** is also known as Equivalence class partitioning (ECP).
- It is a black-box testing that divides input domain into classes of data, and with the help of these classes of data, test cases can be derived.

Example: Consider percentage field that will accept percentage only between 50 to 90 %, more and even less than not be accepted, and application will redirect user to an error page. If percentage entered by user is less than 50 %or more than 90 %, that equivalence partitioning method will show an invalid percentage. If percentage entered is between 50 to 90 %, then equivalence partitioning method will show valid percentage.



notes.arpanadhikari7.com.np

Equivalence Partitioning		
Invalid	Valid	Invalid
<=50	50-90	>=90

Basis Path Testing

- **Basis Path Testing** is a white-box testing technique used to ensure that all independent paths in a program's control flow are executed at least once.
- It focuses on achieving maximum code coverage by identifying test cases that cover all possible paths through a program.
- A control flow graph is prepared and the various possible paths present in the graph are executed as a part of testing.
- To design test cases using this technique, four steps are followed:

1. Construct the Control Flow Graph
2. Compute the Cyclomatic Complexity of the Graph
3. Identify the Independent Paths
4. Design Test cases from Independent Paths

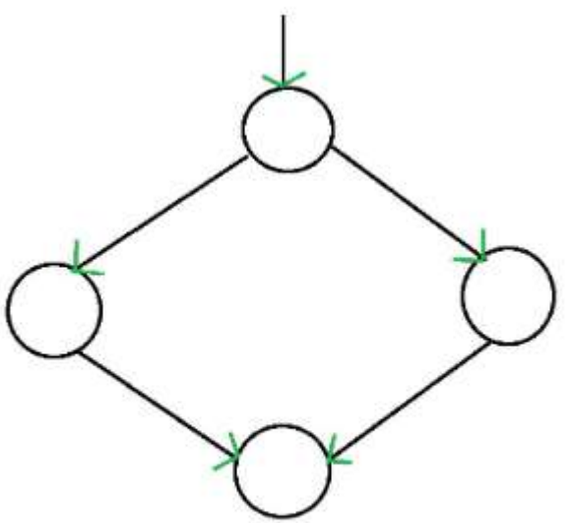
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

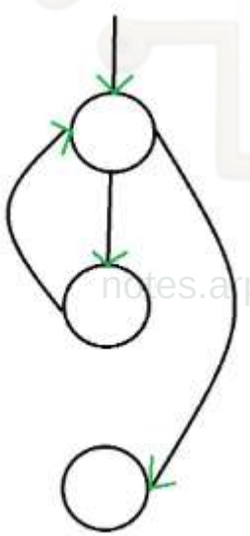
Basis Path Testing (contd…)

Control Flow Graph (CFG)

- A **Control Flow Graph (CFG)** is the graphical representation of control flow or computation during the execution of programs or applications.
- Control flow graphs are mostly used in static analysis as well as compiler applications, as they can accurately represent the flow inside a program unit.
- Control Flow Graph is represented differently for all statements and loops. The following images describe it:



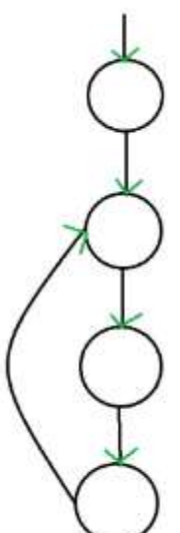
If-then-else



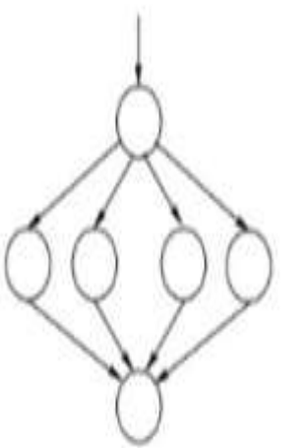
while



do-while



for



case

notes.arpanadhikari7.com.np

Control Structure Testing

- **Control structure testing** is a type of white-box testing technique that focuses on the logical control structures in the source code.
- Its goal is to ensure that every possible execution path, decision, and loop in a program is tested at least once.
- This helps identify errors in logic, flow, and conditions.
- The different types of testing performed under control structure testing are as follows-

1. Condition Testing
2. Data Flow Testing
3. Loop Testing

notes.arpanadhikari7.com.np

Control Structure Testing (contd…)

Conditional Testing:

- **Condition testing** is a white-box testing technique that focuses on evaluating logical conditions within a program to ensure that all possible outcomes of each condition are exercised.
- It helps identify issues with logical operators and ensures that the code behaves as expected under all possible scenarios.
- Type of conditional testing

1. Simple Condition
2. Complex Condition

notes.arpanadhikari7.com.np

Control Structure Testing (contd…)

Data Flow Testing

- **Data flow testing** is a white-box testing technique that focuses on the lifecycle of data variables within a program. It examines how data values are defined, used, and propagated in the code, aiming to detect issues such as uninitialized variables, unused variables, and incorrect data usage.

notes.arpanadhikari7.com.np

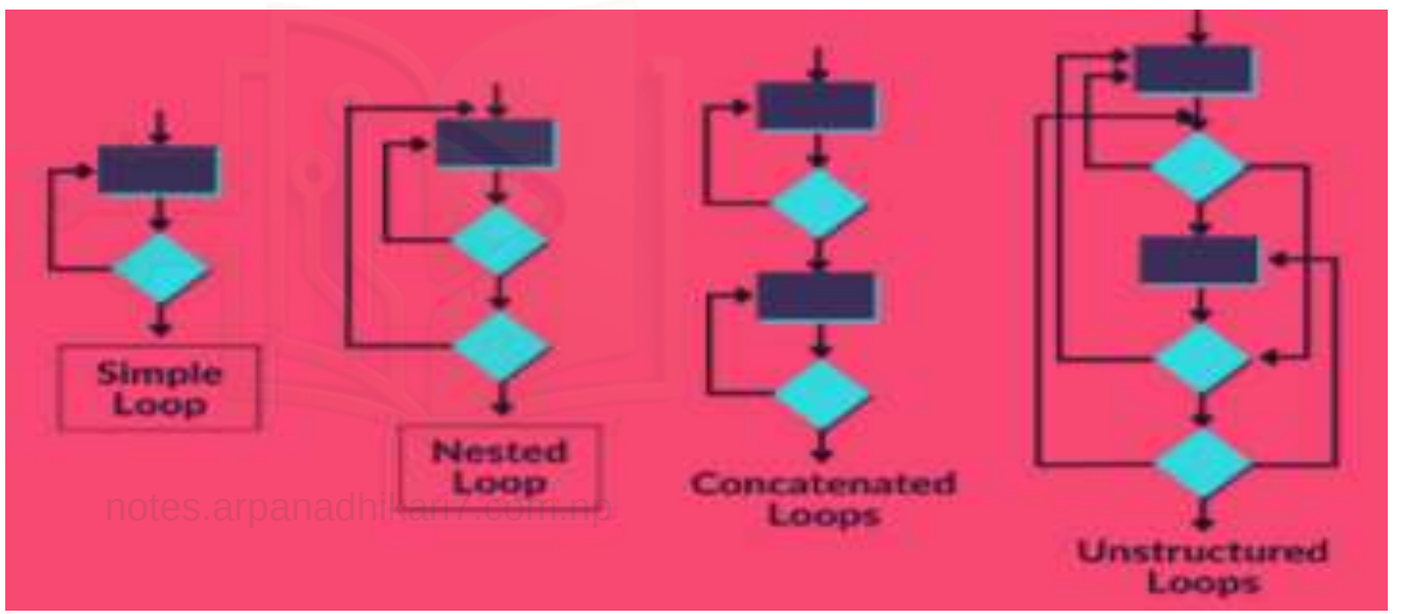
Control Structure Testing (contd…)

Loop Testing

- **Loop testing** is a white-box testing technique that focuses on validating the functionality and correctness of loops in a program. Loops are critical control structures, and errors in their implementation can lead to issues like infinite loops, incorrect iterations, or logic flaws. Loop testing ensures that loops function as intended across all scenarios.

- Types of loop

1. Simple loop
2. Nested loop
3. Concatenated loop
4. Unstructured loop



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Control Structure Testing (contd…)

Simple loop:

The following set of test can be applied to simple loops, where the maximum allowable number through the loop is n.

1. Skip the entire loop.

2. Traverse the loop only once.

3. Traverse the loop two times.

4. Make p passes through the loop where p<n.

5. Traverse the loop n-1, n, n+1 times

notes.arpanadhikari7.com.np

Control Structure Testing (contd…)

Nested loop:

Loops within loops are called as nested loops. when testing nested loops, the number of tested increases as level nesting increases. The following steps for testing nested loops are as follows-

1. Start with inner loop. set all other loops to minimum values.

2. Conduct simple loop testing on inner loop.

3. Work outwards.

4. Continue until all loops tested.

notes.arpanadhikari7.com.np

Control Structure Testing (contd…)

Concatenated loop

If loops are not dependent on each other, contact loops can be tested using the approach used in simple loops. if the loops are interdependent, the steps are followed in nested loops.

Unstructured loop

This type of loops should be redesigned, whenever possible, to reflect the use of unstructured the structured programming constructs

notes.arpanadhikari7.com.np